Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Automatică și Calculatoare
Departamentul Calculatoare și Tehnologia Informației

Computer Architecture
Laborator 10: Calculator Didactic

Ștefan-Dan Ciocîrlan · 2026-2027

Cuprins

1. Unitatea de comandă 1
   1.1. Implementare 2
   1.2. Codificarea instrucțiunilor 2
   1.3. Adresarea directă la registru 4

1. Unitatea de comandă

Ne vom familiariza cu **formatul instrucțiunilor** calculatorului didactic și cu **modul de funcționare al unității de comandă**. În acest scop se vor implementa în Verilog **interpretarea** și **comandarea execuției** pentru instrucțiunile specificate în arhitectura calculatorului didactic studiat la curs.

Componentele calculatorului didactic implementate anterior registre, UAL formează unitatea de execuție a procesorului. Pentru ca acestea să rețină date și să execute instrucțiunile procesorului, avem nevoie de o logică hardware de comandă a acestora, logică implementată în unitatea de comandă.

În interiorul unui procesor, instrucțiunile trec prin mai multe etape. Pentru calculatorul didactic avem următoarele etape:

- **Fetch** - aducerea instrucțiunii din memorie în registrul instrucțiune (RI)
- **Decode** - decodificarea instrucțiunii
- **Load** - încărcarea operanzilor
- **Execute** - executarea instrucțiunii
- **Store** - scrierea rezultatului (dacă este cazul)

Unitatea de comandă este implicată în toate etapele de mai sus: comandă prin semnalare aducerea codurilor instrucțiunilor din memorie, le decodifică și transmite semnale către unitățile (registre, UAL, memorie) implicate în execuția acelor instrucțiuni. La terminarea execuției fiecărei instrucțiuni se comandă scrierea rezultatului (dacă este cazul) și se actualizează registrul **CP** (Contor Program) cu adresa instrucțiunii următoare.

Ce trebuie să facă unitatea de comandă atunci când trebuie executată o instrucțiune aritmetică logică cu un operand? De exemplu `INC RA`.

- Semnale către **CP** și **AM**: valoarea din **CP** e pusă pe magistrală și scrisă în **AM**
- Semnale către **AM** și memorie pentru a lua valoarea (codul instrucțiunii) de la adresa specificată de **AM**
- Semnal către **RAM** pentru a pune valoarea pe magistrală și către **RI** pentru a pune codul instrucțiunii în el;

- Decodificare instrucțiune;
- Semnal către blocul de registre generale pentru a determina activarea conținutului lui RA pe magistrală și semnal către **T1** pentru a încărca valoarea aflată în acest moment pe magistrală;
- Semnal către **UAL** ce indică operația **INC**
- Semnal către **IND** pentru ca UAL-ul să poată scrie flagurile;
- Semnal către blocul de registre generale pentru a încărca în **RA** valoarea de pe magistrală;
- Incrementarea **CP** pentru adresa instrucțiunii următoare - semnal către **CP** pentru a scrie în el.

1.1. Implementare

Unitatea de comandă este implementată ca un **automat de stări**. Modulul acesteia are următoarele semnale:

- intrări: **clk**, **rst**, **ri** (codul instrucțiunii), **ind** (indicatorii de condiție)
- ieșiri:
  - semnale **oe** (output-enable) și **we** (write-enable) pentru registre, bancul de registre, memorie și unitatea aritmetică logică (doar **oe**)
  - **alu_opcode** - codul operației ce trebuie efectuată de unitatea aritmetică logică
  - **alu_carry** - carry-ul folosit de UAL în cadrul operației ce trebuie să o execute
  - **regs_addr** - indexul unui registru din bancul de registre
  - **ind_sel** - controlează sursa de scriere în registrul IND (0 = bus, 1 = alu flags)

Automatul trebuie să ofere stări pentru:

- aducerea instrucțiunii din memorie în registrul **RI**
- decodificarea codului instrucțiunii pentru identificarea operației ce trebuie efectuate și a operanzilor acesteia (dacă este cazul)
- interpretarea fiecărei instrucțiuni. Aceasta se traduce printr-o serie de stări care setează semnalele de output ale modulului pentru a comanda execuția instrucțiunii.
- incrementarea registrului **CP**

În implementarea unității de comandă vom considera că **UAL-ul va pune rezultatul în T1**, și de acolo va fi transferat în registre sau în memorie. Această convenție face mai simple și mai clare stările care comandă execuția operațiilor aritmetico-logice.

1.2. Codificarea instrucțiunilor

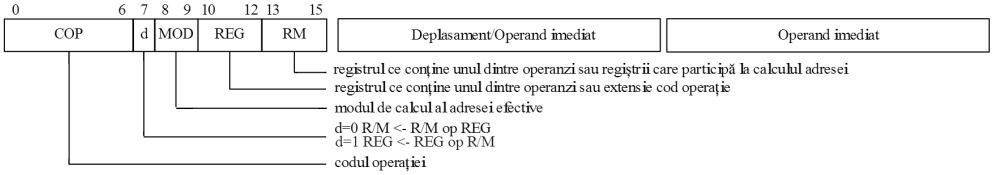


Figura 1: Formatul instrucțiunilor calculatorului didactic

- **COP** - codul operației, 7 biți
  - bitul [0] - separă instrucțiunile care folosesc o adresă efectivă de cele care nu folosesc:
    - 0 - instrucțiuni cu calcul de adresă efectivă
    - 1 - instrucțiuni fără calcul de adresă efectivă (salturi condiționate, RET etc.)
  - bitul [1] - separă instrucțiunile care au 1 operand de cele cu 2 operanzi:
    - 0 - un operand
    - 1 - doi operanzi
  - bitul [2] - separă instrucțiunile cu operand imediat de cele fără operand imediat:
    - 0 - fără operand imediat
    - 1 - cu operand imediat

  - bitul [3] - separă instrucțiunile de transfer de date/control de celelalte:
    - 0 - transfer de date/control (MOV, PUSH, CALL etc.) sau care nu salvează rezultatul (CMP, TEST)
    - 1 - instrucțiuni aritmetico-logice cu salvarea rezultatului, salturi condiționate
  - biții [4][5][6] - cod operație
- **d** - pentru instrucțiunile cu doi operanzi, folosit pentru a ști care e primul și care e al doilea dintre cele două câmpuri REG și RM din codul instrucțiunii:
  - 0 - RM = RM op REG
  - 1 - REG = REG op RM
- **MOD** - modul de calcul al adresei efective (4 moduri - 2 biți)
- **REG** - indexul registrului care conține unul dintre operanzi
- **RM** - indexul registrului sau a adresei de memorie care conține unul dintre operanzi

Tabelul 1: Instrucțiuni aritmetico-logice cu un operand

Instrucțiune	Funcție	Cod RI[0:6]
INC	op = op + 1	0001 000
DEC	op = op - 1	0001 001
NEG	op = -op	0001 010
NOT	op = op	0001 011
SHL/SAL	op = op << 1	0001 100
SHR	op = op >> 1	0001 101
SAR	op = op >>> 1	0001 110

De exemplu, pentru instrucțiunea `INC RA` grupul este cel al operațiilor cu calcul de adresă efectivă (RI[0] = 0), cu un singur operand (RI[1] = 0), fără operand imediat (RI[2] = 0) și cu salvarea rezultatului (RI[3] = 1).

Pentru orice procesor, fiecare instrucțiune definită în arhitectura setului său de instrucțiuni, are un anumit **cod unic după care este identificată**. Atunci când se stochează o instrucțiune care lucrează cu cel puțin un operand, nu este suficient să avem doar codul său (care indică ce acțiune trebuie efectuată), ci trebuie să avem și niște biți care să ne indice de unde luăm operanzii, așa cum este ilustrat și în imaginea de mai jos.

Dacă MOD = 2′b11 (adresare directă la registru) și instrucțiunea are un singur operand, acesta este pus în RM.

Se poate observa că în cadrul unora din grupurile de operații au rămas codificări nefolosite pe biții 4:6. Dacă se extinde setul de instrucțiuni cu noi operații (**vedeți în curs!**), atunci codul acestora poate fi unul din cele nefolosite (atâta timp cât se încadrează în acel grup).

Exemplu de calcul cod operație pentru instrucțiune `DEC RB`:

- [0] = 0 - cu calcul de adresă efectivă
- [1] = 0 - un operand
- [2] = 0 - fără operand imediat
- [3] = 1 - operație aritmetică cu salvarea rezultatului
- [4,5,6] = [0,0,1] - codul dat operației (stabilit de arhitectură)
- [7] = 0/1 - nu contează, îl putem pune 0 sau 1
- [8,9] = [1,1] - modul de adresare (directă la registru)
- [10,11,12] = [0,0,0] - nu folosim REG, deci nu contează ce punem
- [13,14,15] = [0,0,1] - indexul registrului RB în bancul de registre

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	1	1	0	1	0	0	1	0	0	0

Instrucțiunile care vor fi implementate în acest laborator se regăsesc în tabelul de mai jos. Acestea sunt instrucțiuni aritmetice și logice cu doi operanzi ale căror valori sunt în registrele generale și fie pun un rezultat înapoi în registrul destinație, fie nu stochează rezultatul ci doar setează indicatorii de condiție (CMP și TEST).

Tabelul 3: Instrucțiuni aritmetico-logice cu doi operanzi

Instrucțiune	Funcție	Cod RI[0:6]
ADD	$op_{dst} = op_{dst} + op_{src}$	0101000
ADC	$op_{dst} = op_{dst} + op_{src}$ + carry	0101001
SUB	$op_{dst} = op_{dst} - op_{src}$	0101010
SBB	$op_{dst} = op_{dst} - op_{src}$ - carry	0101011
AND	$op_{dst} = op_{dst}$ & op_{src}	0101100
OR	$op_{dst} = op_{dst} \mid op_{src}$	0101101
XOR	$op_{dst} = op_{dst}$ ^ op_{src}	0101110
CMP	$op_{dst} - op_{src}$, fără stocare rezultat, doar setare indicatori	0100010
TEST	op_{dst} & op_{src}, fără stocare rezultat, doar setare indicatori	0100100

Pentru **decodificarea** instrucțiunilor din acest laborator trebuie sa identificăm atât grupul instrucțiunilor aritmetico-logice cu doi operanzi, fără operand imediat și care stochează rezultatul (**$RI_{0..3}$ = 0101**) cât și cele care nu stochează rezultatul (**$RI_{0..3}$ = 0100**).

1.3. Adresarea directă la registru

Operanzii se găsesc în registrele specificate de câmpurile REG și RM. Pentru a selecta ordinea operanzilor din aceste două câmpuri, trebuie să ținem cont de valoarea din bitul **d**. Vom folosi adresarea directă la registru (câmpul mod trebuie să conțină valoarea 2'b11), așa cum se observă în imaginea de mai jos.

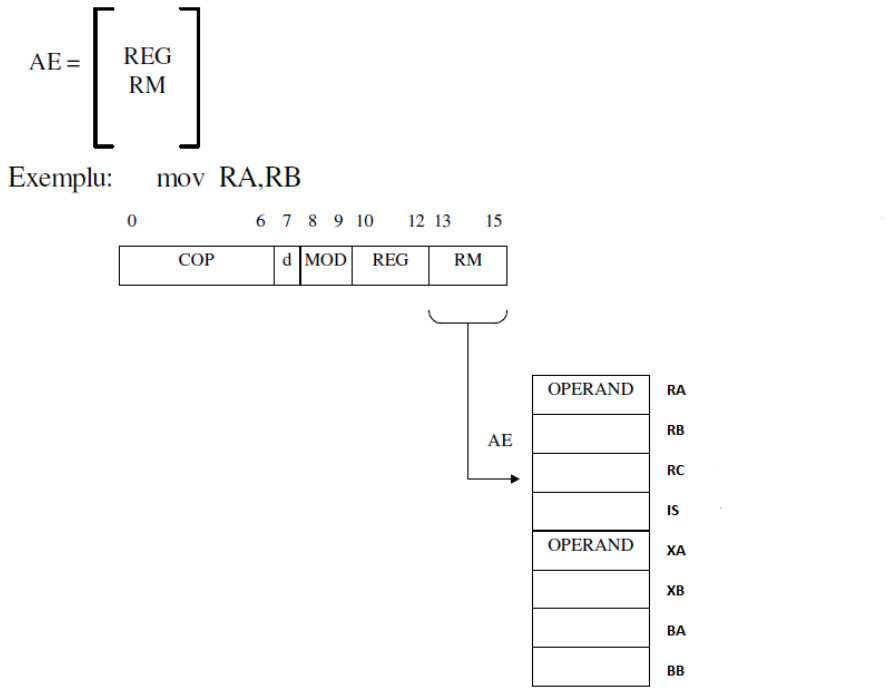


Figura 2: Adresarea directă la registru pentru instrucțiuni cu doi operanzi

Instrucțiunea `AND RA, RB` efectuează `ȘI` logic între cei doi operanzi și pune rezultatul în registrul destinație (`RA`). Un exemplu de calcul al codului operației se regăsește mai jos:

- [0] = 0 - cu calcul de adresă efectivă
- [1] = 1 - doi operanzi
- [2] = 0 - fără operand imediat
- [3] = 1 - operație aritmetică cu salvarea rezultatului
- [4,5,6] = [1,0,0] - codul dat operației (stabilit de arhitectură)
- [7] - selectează operandul destinație. Pentru exemplul acesta vom considera că acest bit ia valoarea 1 (ținem cont că destinația va fi registrul `RA`):
  - 1: REG = REG `AND` RM
- [8,9] = [1,1] - modul de adresare (folosim adresarea directă la registru)
- [10,11,12] = [0,0,0] - indexul registrului `RA` în bancul de registre
- [13,14,15] = [0,0,1] - indexul registrului `RB` în bancul de registre

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	1	1	1	0	0	1	1	0	1	0

O altă variantă pentru codificarea acestei operații apare în urma modificarii bitului [7] astfel:

- [7] - selectează operandul destinație. Pentru exemplul acesta vom considera că acest bit ia valoarea 0 (ținem cont că destinația va fi registrul `RA`):
  - 0: RM = REG `AND` RM
- [10,11,12] = [0,0,1] - indexul registrului `RB` în bancul de registre
- [13,14,15] = [0,0,0] - indexul registrului `RA` în bancul de registre

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	0	0	1	1	0	0	0	1	1	0	1	0

Fișierul care conține cheat-sheet-ul cu ajutorul căruia puteți decodifica orice instrucțiune poate fi găsit aici

Fișierul care descrie intreaga arhitectură este aici: Biblia.pdf